AI for Novel Blackjack

Meihan Zheng[1], **Tianyu Gu**[1], **Yatu Zhang**[1]

[1]School of Information Science and Technology, ShanghaiTech University
Student IDs: 2023533018 (Meihan Zheng), 2023533051 (Tianyu Gu), 2023533005 (Yatu Zhang)
zhengmh2023@shanghaitech.edu.cn, guty2023@shanghaitech.edu.cn, zhangyt2023@shanghaitech.edu.cn

Abstract

In this project, we develop multiple agents for a novel Blackjack with expanded actions (hit, stay, switch, fold) and dynamic betting rules, leveraging techniques from reinforcement learning and decision theory. We implement Random, Expectimax, Markov Decision Process (MDP), Q-learning, and Deep Q-Network (DQN) agents, alongside a New Q-learning agent with a refined reward function. We design a comprehensive evaluation framework to assess win rates and betting efficiency in a simulated casino environment. By bridging theoretical AI concepts with practical implementation, this project enhances strategic decision-making insights and demonstrates the applicability of AI in complex gaming scenarios, illustrating AI's potential in gaming and beyond.

Introduction

This project focuses on the game of blackjack, which is a casino banking game in which players compete against the dealer. Since there is a lot of research that focuses on AI methods in classic blackjack, we invented a brand new blackjack game with new play rules and new bet rules. This new blackjack offers an innovative testbed for AI algorithms while demonstrating our contributions in the meantime.

Play Rule

Players can choose one of the following actions:

- Hit: Draw an additional card.
- Stay: Stop drawing cards (ends player's round).
- Switch: Swap cards with the dealer (ends player's round).
- Fold: Hide one of the dealer's cards (ends player's round).

Dealer's rule

- The dealer takes action after the player's turn ends.
- If the dealer's point total is less than 17, he must draw additional cards.

Bet Rule

- Player starts with 100,000 in funds. Initial bet ratio a = 10% of current funds.
- If player has an Ace or 10 (including J, Q, K), multiply a by 1.5.
- If dealer's upcard is an Ace or 10 (including J, Q, K), multiply a by 0.75.
- If player loses 5 consecutive rounds, multiply a by 0.75.
- If player wins 5 consecutive rounds, multiply a by 1.5.
- Each round, multiply a by a random value in the range [0.9,1, 1.1].

State Space

In our approach, the state space is designed to encapsulate the essential information required to evaluate the current game situation. Each state is represented by a tuple that captures key aspects of the game at any given decision point. The state is defined as:

$$s = (\text{PlayerVal}, \text{DealerVal}, \text{HasAce})$$

The components of the state tuple are:

1. **PlayerVal**: The sum of the player's current hand, calculated as the total value of the cards held by the player. This value ranges from 2 to 21, or exceeds 21 if the player busts.
2. **DealerVal**: The value of the dealer's upcard, which is the single card visible to the player at the start of a decision round. This value typically ranges from 2 to 11 (considering an Ace as 11).
3. **HasAce**: A boolean indicator representing whether the player's hand contains a usable ace. A usable ace is one that can be counted as 11 without causing the player's hand to exceed 21.

The state space is designed to be finite and manageable, facilitating efficient decision-making in various strategies.

Motivation

We implement a range of agents, including random agents, adversarial search agents using expectimax, Markov Decision Process-based agents, Q-learning agents, and Deep Q-learning Network (DQN) agents.

To evaluate the performance of these agents, we adopt a two-stage comparison process:

1. **Stage 1: Win Rate Comparison**:We assess the agents' success rates in winning games against the dealer.

2. **Stage 2: Bets Comparison**:We analyze the agents' bet efficiency and decision-making under our bet rule.

Through this two-stage comparison of different agents, we derive conclusions about the effectiveness of different methods and our reward function in capturing the strategic nuances of Blackjack.

Contribution

Our contributions are as follows:

- We design a brand-new Blackjack game that incorporates four distinct actions (hit, stand, double down, and split), expanding the strategic complexity beyond traditional Blackjack implementations.
- We develop and evaluate five distinct agents, including random, adversarial search (expectimax), Markov Decision Process, Q-learning, and Deep Q-learning Network agents—to explore a wide range of decision-making strategies.
- We implement a comprehensive win-rate calculation to quantitatively assess and compare the performance of each agent, providing clear insights into their strategic effectiveness.
- We conduct a casino simulation to compare the agents' performance. This simulation highlights the impact of betting decisions on overall performance, offering a practical perspective on the agents' applicability in casino-like environments.
- We try to find out the reason why q-learning agent sometimes cannot converge in the real casino simulation. We visualize the q-table value and invent an advanced q-learning agent with proper reward function and improve the performance of original q-learning.

Methods

In this part we will introduce the five agents we have done.

Score Function

In our methods discussed in the introduction, most approaches require a score function to evaluate the action. Consequently, designing a reasonable and effective score function is crucial. Across various methods, the factors influencing the score function are consistent. The key factors are:

1. **Player busts:** If the player busts (exceeds a sum of 21), the score is -1.
2. **Dealer busts (Player does not bust):** If the dealer busts while the player does not, the score is $+1$, indicating a win for the player.
3. **PlayerVal greater than DealerSum (no bust):** When neither the player nor the dealer busts, and the player's card sum exceeds the dealer's, the score is $+1$, signifying a win.
4. **PlayerVal less than DealerSum (no bust):** If the player's sum is less than the dealer's without either busting, the score is -1, indicating a loss.
5. **Equal sums:** If the player's and dealer's sums are equal without busting, the score is 0, representing a draw.

The score function we employ is defined as:

$$f(s) = \begin{cases} -1 & \text{Player busts} \\ +1 & \text{Dealer busts and Player does not bust} \\ +1 & \text{PlayerVal} > \text{DealerSum (no bust)} \\ -1 & \text{PlayerVal} < \text{DealerSum (no bust)} \\ 0 & \text{PlayerVal} = \text{DealerSum(no bust)} \end{cases}$$

This score function effectively captures the outcome of each game state, enabling consistent evaluation across different decision-making strategies in our methods.

Random

This is a naive but fast agent. We randomly choose one of the four player's action (hit, stay, switch, fold) and perform the action. Although random agents perform very poor in classic blackjack, its performance is not so bad in our new blackjack due to the special play rule. Therefore, it is an ideal agent for fast testing with other types of agents.

Expectimax

As a two-player game, Blackjack can be effectively approached using adversarial search techniques. In this work, we implement an expecimax search algorithm to determine optimal actions for the player. The expectimax algorithm is implemented recursively, evaluating the game state at each step to select the action that maximizes the player's expected outcome while considering the dealer's expected response. State values under player's control are expressed as follows:

$$V(s) = \max_{s' \in \text{successors}(s)} V(s')$$

State values under dealer's control are expressed as follows:

$$V(s') = \sum_{s \in \text{successors}(s')} P(s) \times V(s)$$

To ensure computational efficiency and limit search time, we restrict the maximum search depth. When the search reaches the depth limit, the reward function is used to estimate the outcome and select the best move.

To further optimize the search process, we incorporate alpha-beta pruning, reducing the number of nodes explored in the search tree. This optimization allows the algorithm to explore deeper within the same computational constraints, thereby improving decision-making performance without sacrificing accuracy and making it a robust solution for Blackjack.

Markov Decision Process

The Markov Decision Process (MDP) framework provides a robust and principled approach for decision-making. MDP

is well-suited for sequential decision-making under uncertainty in BlackJack, explicitly modeling the stochastic nature of the environment.

We implemented two distinct MDP agent to evaluate their impact on the learning process. In the basic MDP agent, the state value table is represented as a three-dimensional array of size $32 \times 12 \times 2$, storing the expected return for each state. To account for bankroll management, we extend the state space to include betting context, creating an MDPWithBet agent. The extended state includes two additional dimensions:

- **Bank_Norm**: The normalized bankroll level, computed as the current bankroll divided by an initial reference bankroll (B/B_0). This ratio allows the agent to adapt its strategy based on available capital.
- **A_Norm**: The normalized bet ratio, defined relative to a base bet fraction a_0. The betting fraction a is calculated each round based on bet rules and normalized by a_0.

For each state s, $Q(s, a)$ is calculated as the expected return of taking action a, considering the transition probabilities $p(s', r \mid s, a)$, immediate reward r, and the discounted future value $\gamma V(s')$, where γ is the discount factor. $r \in \{-1, 0, +1\}$ reflects whether the player eventually busts (-1), draws (0), or wins ($+1$) for a single unit bet. To incorporate bankroll impact, we scale this by the normalized bet size in that round. We update the reward and bankroll as follows:.

$$r' = r \times (\text{Bank\_Norm} \times \text{A\_Norm})$$

$$\text{Bank\_Norm '} = \text{Bank\_Norm } \times (1 + r' \times \text{A\_Norm})$$

Both Bank_Norm and A_Norm are kept within bounded, discretized ranges. This prevents numerical explosion in the value iteration and ensures stability while still reflecting that larger bets lead to larger absolute gains or losses.

Value Iteration We use value iteration to update the state value table $V(s)$ to compute the expected return for each state until convergence. The value function is updated as follows:

$$V(s) \leftarrow \max_{a \in A} Q(s, a)$$

The Q-value $Q(s, a)$ estimates the expected return when taking action a in state s and then following the optimal policy.

The Q-value updates for each action are defined as follows:

$$Q(s, \text{Stay}) = \sum_{\text{DealerSum}} p(\text{DealerSum} \mid \text{DealerVal}) \cdot r$$

$$Q(s, \text{Hit}) = \sum_{c \in \text{cards}} p(c) \cdot \begin{cases} r & \text{PlayerVal} > 21 \\ \gamma V(s') & \text{otherwise} \end{cases}$$

$$Q(s, \text{Fold}) = \sum_{c \in \text{cards}} p(c) \cdot \gamma \cdot V(s')$$

$$Q(s, \text{Switch}) = \sum_{c \in \text{cards}} p(c) \cdot \begin{cases} r & \text{PlayerVal} > 21 \\ \gamma \cdot V(s') & \text{otherwise} \end{cases}$$

Policy Extraction The optimal policy is extracted by selecting the action that maximizes $Q(s, a)$ for each state, stored in the policy table. The optimal policy is extracted as follows:

$$\pi(s) = \arg\max_{a \in A} Q(s, a)$$

During gameplay, the agent queries the policy table to determine the optimal action for the current state. MDP's ability to model uncertainty, optimize long-term rewards, handle complex actions, and provide computational efficiency make it an ideal choice for extracting an optimal strategy in BlackJack.

Q-learning

Q-learning is a model-free reinforcement learning algorithm that learns an optimal policy by iteratively updating q-value estimates. The algorithm maintains a Q-table of size $32 \times 12 \times 2 \times 4$ that stores estimated rewards for each state-action pair.

Reward Function We implemented two distinct reward functions to evaluate their impact on the learning process. The first reward function, $R_1(s, a, s')$, is defined as follows:

$$R_1(s, a, s') = \begin{cases} -1 & \text{Player busts} \\ +1 & \text{Dealer busts and Player does not bust} \\ +1 & \text{PlayerVal} > \text{DealerSum (no bust)} \\ -1 & \text{PlayerVal} < \text{DealerSum (no bust)} \\ 0 & \text{PlayerVal} = \text{DealerSum (no bust)} \\ +0.1 & \text{Action = Switch} \\ +0.1 & \text{Action = Fold} \end{cases}$$

The second reward function, $R_2(s, a, s')$, incorporates additional conditions based on the player's sum and action taken, aiming to incentivize strategic decisions:

$$R_2(s, a, s') = \begin{cases} -1 & \text{Player busts} \\ +1 & \text{Dealer busts and Player does not bust} \\ +1 & \text{PlayerVal} > \text{DealerSum (no bust)} \\ -1 & \text{PlayerVal} < \text{DealerSum (no bust)} \\ 0 & \text{PlayerVal} = \text{DealerSum(no bust)} \\ +0.8 & \text{Action = Switch and PlayerVal} >= 15 \\ -0.1 & \text{Action = Switch and PlayerVal} < 15 \\ +0.8 & \text{Action = Fold and PlayerVal} >= 15 \\ -0.1 & \text{Action = Fold and PlayerVal} < 15 \\ +0.9 & \text{Action = Hit and PlayerVal} < 10 \\ +0.3 & \text{Action = Hit and } 10 <= \text{PlayerVal} < 15 \\ -0.5 & \text{Action = Hit and PlayerVal} >= 15 \end{cases}$$

Q-value Iteration Considering the training, when we take an action, we have a new sample and we use the sample to update our Q-value. The procedure is as follows:

$$sample = R(s, a, s') + \gamma \max_{a'} Q(s', a')$$

$$Q(s, a) \leftarrow (1 - \alpha) Q(s, a) + \alpha (sample)$$

Policy Extraction As for policy selection, we use ϵ-greedy strategy, and set ϵ as 0.1. Therefore, we can have 0.1 probability randomly choose an action, and 0.9 probability choose an action according to the Q-table, which balance the exploration and exploitation properly.

DQN

DQN represents an advancement over Q-learning by leveraging neural networks to approximate the Q-value function. Unlike tabular Q-learning, which stores Q-values for each state-action pair, DQN uses a neural network to generalize across states in Blackjack.

Q-Network We employ a multilayer perceptron (MLP) parameterized by θ to approximate $Q_\theta(s, a)$, producing a Q-value for each possible action given the state vector. To stabilize training, a separate target network Q_{θ^-} is maintained; its weights are periodically updated by copying from the main network. This decoupling reduces oscillations when computing the Bellman target.

Bellman Update For each sampled transition $(s, a, r, s', \text{done})$, we form the Bellman target:

$$y = \begin{cases} r, & \text{if terminal} \\ r + \gamma \max_{a'} Q_{\text{target}}(s', a'), & \text{otherwise} \end{cases}$$

The learning objective minimizes the mean squared error:

$$L(\theta) = \mathbb{E}_{(s,a,r,s')} \left[(y - Q_\theta(s, a))^2 \right]$$

We then backpropagate this loss on minibatches drawn from replay to update θ.

Experience Replay We store transitions in a replay buffer. During training, we sample random minibatches to break temporal correlations and improve sample efficiency. We then use ε-greedy exploration to balance between exploration and exploitation.

By combining a Q-network with experience replay and a target network, DQN achieves stable and scalable learning across richer state representations, offering clear advantages over traditional tabular Q-learning in Blackjack when state features are numerous.

Experiments

We conduct two experiments on the different agents we design and try to analyze the underlying reason for their differing performances in our game.

Win Rate Comparison

We first let each agent play the Blackjack game 10000 times and calculated the win rate of each agent. We visualized the win rate of different agent in Figure 1, showing the agents ranked from lowest to highest win rate: Random, Expectimax, Q-learning, New Q-learning, DQN, and MDP.

To understand the performance disparities among the agents, we analyze why the MDP agent performs best, why the DQN agent surpasses the Q-learning agent, and why the Expectimax agent exhibits a lower win rate.

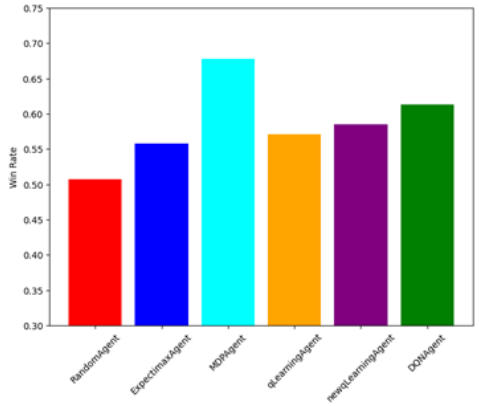

Figure 1: Win Rate Comparison

The MDP agent achieves superior performance due to its model-based approach, leveraging explicit transition probabilities and Blackjack's reward structure to compute the optimal policy via value iteration over the state space. Additionally, MDP's Bellman updates compute precise expected returns.

The DQN agent outperforms the Q-learning agent by using a neural network to generalize Q-values across similar states, enabling faster learning and noise reduction compared to tabular Q-learning, which struggles with sparse state visits. DQN's replay buffer improves sample efficiency by reusing past transitions and breaking temporal correlations, while its target network stabilizes updates and reduces oscillations.

The Expectimax agent underperforms due to its reliance on shallow lookahead with limited depth, resulting in a lower win rate compared to agents with optimized policies.

Bet Comparison

To evaluate the agents in a realistic casino setting, we simulated 300 rounds of Blackjack with betting, tracking each agent's bankroll performance. Figure 2 illustrates the bankroll growth curves, showing that the MDPWithBet and DQN agents converge fastest to the maximum possible earnings, followed by the MDP and New Q-learning agents. However, the Expectimax and Random agents fail to converge. We analyze why MDPWithBet outperforms the standard MDP and why the New Q-learning agent outperforms traditional Q-learning agent.

The MDPWithBet agent converges faster than the standard MDP because it incorporates bankroll and dynamic bet sizing into its state and reward structure. Unlike the standard MDP, which optimizes per-round win/loss, MDPWithBet uses value iteration to maximize long-term bankroll growth. This enables risk-sensitive strategies, such as conservative betting when funds are low or aggressive betting during favorable streaks, aligning with real-world casino scenarios.

The New Q-learning agent surpasses the traditional Q-learning agent due to its refined reward function, which better captures the strategic value of actions . As shown

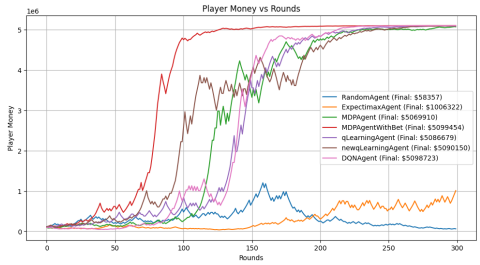

Figure 2: Bet Comparison

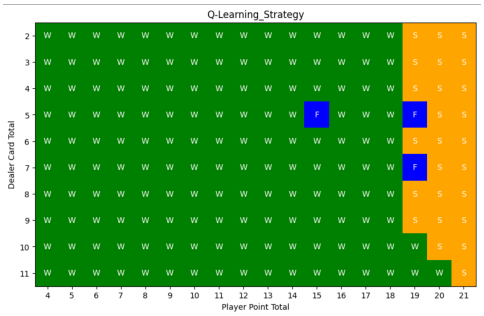

Figure 3: Q-table value of original Q-learning agent

in Figure 4, the Q-table of the New Q-learning agent exhibits greater action diversity, with all four actions utilized across different states, compared to the traditional Q-learning agent's Q-table in Figure 3, which is limited to switch, fold, and stay. This enhanced reward structure allows the New Q-learning agent to make more nuanced decisions, leading to a slightly higher win rate, though challenges remain in achieving full convergence in the casino simulation.

However, the win rate of this agent performs only slightly better than the original Q-learning agent. We will improve this agent and try to solve this problem in the later work.

Conclusion

We introduced a novel Blackjack with expanded actions (hit, stay, switch, fold) and dynamic betting rules, providing a challenging testbed for evaluating AI-driven decision-making strategies. We developed and assessed five agents, including Random, Expectimax, MDP, Q-learning, and DQN, through a two-stage evaluation focusing on win rates and betting efficiency in a simulated casino environment. Our results demonstrate that the MDPWithBet and DQN agents outperform others, with MDPWithBet achieving the fastest convergence to maximum earnings due to its bankroll-aware policy optimization. Additionally, by introducing New Q-learning agent with a refined reward function, we improved the performance of original Q-learning agent. Our work paves the way for future enhancements in state representation and reward design to further improve convergence and performance.

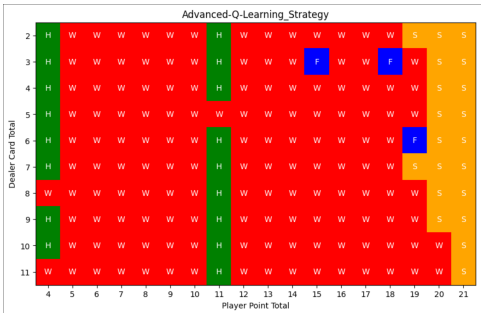

Figure 4: Q-table value of New Q-learning agent

References

KesonStar. 2025. BlackJack. https://github.com/KesonStar/BlackJack. Accessed: 2025-06-13.